



Step-by-Step Instructions

Step 1: Mandatory Technical Requirements

Before sourcing or drafting a task, confirm it fits **all** of the following:

1. The primary programming language is **Kotlin**.
2. **Modern Android practices** (e.g., Jetpack Compose) is implemented.
3. The task covers a **broad range of technology**: Compose (UI), Coroutines (concurrency), Jetpack libraries (Room, WorkManager, Datastore, ViewModel, Hilt), build.gradle(.kts) changes, networking (OkHttp, Retrofit, Coil), and media (CameraX, ExoPlayer, Media3), etc.
Plan distribution, don't cluster in one area.

Step 2: Sourcing via suitable Repositories and issues

1. Prefer **open-source GitHub repositories with a permissive license** (e.g., Apache-2.0). Manually created tasks are acceptable if they meet all criteria.
2. Filter for **realistic, professional Android challenges**; avoid trivial or contrived examples.
3. Record for each selected repo: canonical URL and **exact commit SHA** that will serve as the task's starting point.

Step 3: Problem and Solution Pairs

Construct self contained Problem and Solution pairs.

Step 3a. The Issue

Write a concise, user-facing problem statement (bug or feature) that reflects a real world scenario. Include minimal, necessary context (repro steps, observed vs. expected behavior, environment if relevant).

Step 3b. The Solution

Implement the fix or feature against the specified commit SHA. Keep the patch targeted (avoid unrelated refactors). Ensure it aligns with Modern Android patterns (e.g., Compose UI, Coroutines, Jetpack best practices) where relevant.

Step 4: Verification tests and Gradle command

Add **programmatic tests** that fail before the fix and pass after. Prefer **instrumentation** when appropriate; ensure tests aren't tightly coupled to your exact implementation.

Record the **exact gradle command(s)** to run only the relevant tests (examples below).

Perform a [Self QA](#) before submitting your task.
